

DA5402 – MLOps for Data Science

Indian Institute of Technology Madras

AQI Personal Health Risk Assessment

*A production-grade AI application for
personalized air quality risk classification
across Indian cities*

Name: Jaydeep Makwana

Roll No: DA25M013

Program: M.Tech, Data Science and AI

Course: DA5402 – MLOps

April 2026

Contents

1	Problem Statement	3
1.1	Success Metrics	3
2	System Architecture	3
2.1	Component Overview	3
2.2	Architecture Diagram	4
2.3	Data Flow	4
3	ML System Design	4
3.1	Model Choice	4
3.2	Features	5
3.3	Target Classes	5
3.4	Evaluation Metrics	5
4	Software Engineering	5
4.1	Design Documents	5
4.2	API Endpoints (Low-Level)	5
4.3	Module Structure	6
4.4	Loose Coupling	6
4.5	Coding Standards, Logging, Exception Handling	6
5	MLOps Implementation	7
5.1	Data Engineering – Apache Airflow	7
5.2	Source Control & CI – DVC + Git	7
5.3	Experiment Tracking – MLflow	8
5.4	Drift Detection & Auto-Retraining	8
5.5	Prometheus Instrumentation	8
5.6	Grafana Visualization	9
5.7	Software Packaging – Docker Compose	9
5.8	Rollback Strategy	9
6	Frontend (Streamlit)	9
6.1	Risk Assessment Page	9
6.2	Pipeline Monitor Page	9
6.3	User Manual	10
7	Testing	10
7.1	Test Plan	10
7.2	Test Case Inventory	10
7.3	Acceptance Criteria	10
8	Results	10
9	Screenshots	11
9.1	Frontend	11
9.2	Apache Airflow	12
9.3	MLflow	14
9.4	Prometheus	15
9.5	Grafana	16
9.6	DVC, Metrics, and Docker	17

10 Conclusion	17
10.1 Known Limitations and Future Work	17

1 Problem Statement

Indian cities routinely experience hazardous air quality, yet existing public tools (CPCB SAMEER, government dashboards) only report a raw AQI number bucketed into a static color band. This is information, not advice. The same AQI of 220 means very different things for a 60-year-old asthmatic planning a long walk versus a healthy 25-year-old going to a meeting indoors. Personal context is missing.

This project builds an end-to-end ML application that closes that gap. The system takes a user’s location, current pollution readings (AQI, PM2.5, PM10, NO₂), local weather (temperature, humidity, wind speed), and a small health profile (age group, pre-existing conditions, planned activity intensity), and returns one of three actionable tiers: **Safe**, **Caution**, or **Avoid Outdoors**, along with a written recommendation and a confidence score.

The MLOps angle is what makes this realistic rather than a notebook exercise. Indian air quality data is structurally non-stationary: the monsoon, Diwali firecrackers, and Punjab stubble burning each cause violent, predictable distribution shifts that would silently degrade any static model. The system therefore ingests fresh CPCB data hourly, runs a Kolmogorov–Smirnov drift test on every batch, and triggers an automated retraining DAG when drift is detected, all without manual intervention.

1.1 Success Metrics

- **ML metric:** F1-Macro ≥ 0.80 on a held-out test set (chosen over accuracy because the three risk tiers are imbalanced — Safe is far more common than Avoid Outdoors).
- **Business metric:** P95 inference latency < 200 ms.
- **Operational metric:** Drift-triggered retraining loop completes end-to-end without human intervention.

2 System Architecture

The application is a decoupled, fully containerized microservice system. The frontend and backend communicate *only* over REST — the frontend has no direct access to the model, the database, or any data store. This is the loose coupling required by the design rubric and also lets the two services be scaled, deployed, and reasoned about independently.

2.1 Component Overview

Service	Technology	Responsibility
Frontend	Streamlit (port 8501)	Form input, results display
Backend	FastAPI (port 8000)	REST API, inference, metrics
Pipeline	Apache Airflow (port 8080)	Hourly ingestion, retraining
Tracking	MLflow (port 5000)	Experiments, model registry
Monitoring	Prometheus (port 9090)	Metrics scraping
Dashboard	Grafana (port 3000)	Visualization, alerting
Versioning	DVC + Git	Code, data, model versions
DB (internal)	PostgreSQL	Airflow metadata

Table 1: Service inventory and responsibilities

2.2 Architecture Diagram

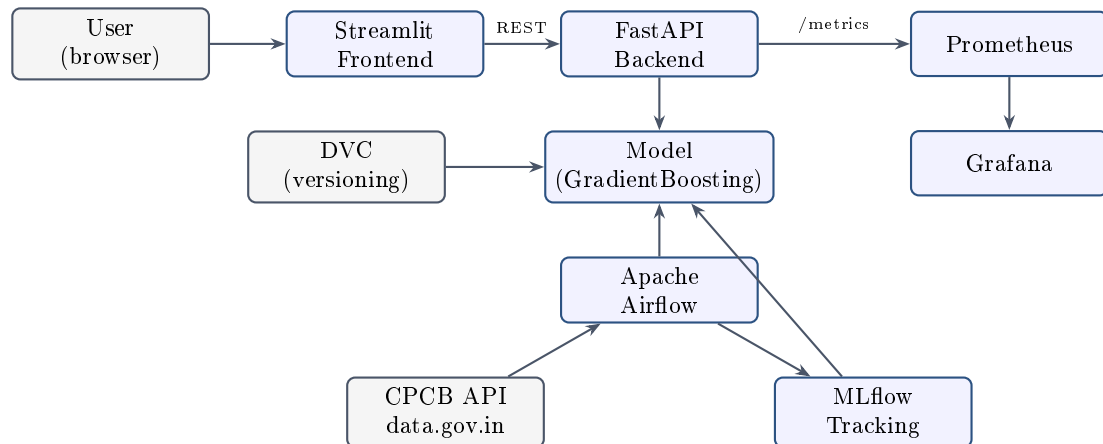


Figure 1: High-level service architecture. The frontend never touches the model directly; all access goes through the FastAPI REST layer.

2.3 Data Flow

1. Airflow’s `cpcb_data_ingestion` DAG fetches hourly AQI data from the CPCB API.
2. Data is validated (schema, null rates), cleaned, outliers removed, and features engineered.
3. A KS-test compares the current window against the training-time baseline.
4. If drift is detected, `model_retraining` DAG is triggered; otherwise the run ends.
5. The retraining DAG generates fresh data, preprocesses, trains with MLflow tracking, evaluates, and registers the new model in the MLflow registry as a new *Production* version.
6. The backend reloads the production model on the next health-check cycle.
7. User submits a form via Streamlit; backend runs inference, returns risk tier; Prometheus records the prediction; Grafana visualizes it in near-real-time.

3 ML System Design

3.1 Model Choice

A `GradientBoostingClassifier` from scikit-learn was chosen after weighing alternatives. The reasoning:

- Handles mixed continuous and categorical features natively without elaborate encoding pipelines.
- Achieves strong accuracy on small-to-medium tabular datasets without being hyperparameter-sensitive — important because the model is retrained automatically and we cannot afford manual tuning per retrain.
- Produces well-calibrated probabilities, which feeds directly into the user-facing confidence score.
- Trains in seconds on commodity CPU. Given the “no cloud” constraint, this matters: the retraining DAG must complete within the Airflow scheduler window.
- Feature importances are directly available, which supports the explainability requirement.

A neural network was considered and rejected: the dataset is small, the features are tabular and low-dimensional, and a NN would add training time, GPU dependency, and interpretability cost without measurable accuracy benefit.

3.2 Features

The feature vector has 11 dimensions, fed through a `StandardScaler` before the classifier:

Feature	Type	Description
aqi	float	Overall AQI (0–500)
pm25	float	PM2.5 concentration ($\mu\text{g}/\text{m}^3$)
pm10	float	PM10 concentration ($\mu\text{g}/\text{m}^3$)
no2	float	NO ₂ concentration ($\mu\text{g}/\text{m}^3$)
temperature	float	Celsius
humidity	float	Relative humidity (%)
wind_speed	float	km/h
age_group_enc	int 0–3	child / teen / adult / senior
has_asthma	0/1	Respiratory condition flag
has_heart_condition	0/1	Cardiovascular flag
planned_activity_enc	int 0–2	low / moderate / high

Table 2: Model input feature vector

3.3 Target Classes

Class	Label	Meaning
0	Safe	Normal outdoor activity acceptable
1	Caution	Sensitive individuals should limit exposure
2	Avoid Outdoors	Hazardous; stay indoors

Table 3: Risk-tier target classes

3.4 Evaluation Metrics

F1-Macro is the primary metric. Macro-averaging treats each class equally regardless of frequency, which is what we want: the rare *Avoid Outdoors* class is also the most consequential to predict correctly. Accuracy and F1-Weighted are tracked as secondaries. Inference latency is tracked as a business metric with a P95 target under 200 ms.

4 Software Engineering

4.1 Design Documents

The repository ships three design documents under `docs/`: `architecture.md` (system-level architecture and data flow), `HLD.md` (high-level design with model choice rationale and pipeline structure), and `LLD.md` (low-level design with full API endpoint specifications, module structure, data schemas, environment variables, and the exception handling matrix).

4.2 API Endpoints (Low-Level)

The backend exposes the following endpoints. All request and response bodies are validated by Pydantic schemas; the OpenAPI spec is auto-generated and served at `/docs`.

Method	Path	Purpose
POST	/api/v1/predict	Risk-tier inference from city + readings + profile
GET	/api/v1/health	Liveness, model-loaded flag, version
GET	/api/v1/ready	Readiness for orchestration probes
GET	/api/v1/pipeline/status	Last ingestion, drift state, prediction count
POST	/api/v1/feedback	Log ground-truth labels for decay tracking
GET	/metrics	Prometheus exposition format

Table 4: Backend REST API surface

Sample /predict request:

```
{
  "city": "Delhi", "aqi": 280, "pm25": 145.0, "pm10": 220.0, "no2": 78.0,
  "temperature": 22.0, "humidity": 65.0, "wind_speed": 8.0,
  "age_group": "senior", "has_asthma": true,
  "has_heart_condition": false, "planned_activity": "moderate"
}
```

Sample response:

```
{
  "city": "Delhi",
  "risk_tier": "Avoid Outdoors",
  "risk_score": 2,
  "confidence": 0.99,
  "recommendation": "Air quality is hazardous. Avoid all outdoor activities."
},
  "model_version": "ed71e1b6"
}
```

4.3 Module Structure

The backend follows a clean layered structure: `main.py` mounts the application; `core/` holds settings and logging configuration; `models/schemas.py` holds Pydantic models; `api/routes/` holds the four route handlers; `services/` holds the model service, drift service, and Prometheus instrumentation. The `ml/` directory holds standalone training scripts (`generate_data.py`, `train.py`, `evaluate.py`, `drift_detection.py`). The `data_pipeline/` directory holds Airflow DAGs and ingestion scripts. The frontend is a single-file Streamlit multipage app.

4.4 Loose Coupling

The frontend is a Streamlit container running on port 8501. The backend is a FastAPI container on port 8000. The frontend's only knowledge of the backend is the configurable `BACKEND_URL` environment variable. Each service has its own `requirements.txt`, its own Dockerfile, and is built and started independently by Docker Compose. Either can be replaced wholesale (e.g. swapping Streamlit for React) without touching the other.

4.5 Coding Standards, Logging, Exception Handling

The codebase follows PEP 8 with type hints throughout. Logging is configured centrally via `core/logging_config.py` and uses a structured format: `timestamp | level | logger | message`, so logs are grep-able and parseable. Exception handling is explicit and tabulated:

Exception	HTTP	Behaviour
Pydantic ValidationError	422	Auto-handled by FastAPI
RuntimeError (no model)	503	Logged, returned cleanly
Unhandled Exception	500	<code>logger.exception</code> , returned
Requests Timeout (Airflow)	–	Task retries, max 2
CPCB API failure	–	Falls back to synthetic generator

Table 5: Exception handling matrix

5 MLOps Implementation

5.1 Data Engineering – Apache Airflow

Two DAGs run inside the Airflow container. The Airflow webserver is exposed at port 8080 and uses PostgreSQL for metadata. Both DAGs are visible and currently active in the cluster (Figure 4).

`cpcb_data_ingestion` runs hourly (@hourly). Its task graph is:

```
fetch_data → validate_data → preprocess_data → check_drift → {drift_detected,
no_drift} → done
```

`check_drift` is a `BranchPythonOperator` — if the KS-test p-value falls below 0.05 on any monitored continuous feature, the branch routes to `drift_detected`, which in turn triggers the retraining DAG. Otherwise it routes to `no_drift` and ends cleanly. Figure 5 shows the live graph.

`model_retraining` is event-triggered, not scheduled. Its graph is:

```
generate_data → preprocess_data → train_model → evaluate_model → register_model
```

The final task transitions the new model version to the *Production* stage in the MLflow registry. Figure 6 shows it.

Pipeline throughput. A full hourly ingestion run completes in roughly 30–90 seconds end-to-end on local hardware. The retraining DAG completes in 5–7 minutes (the bottleneck is data generation when the CPCB API is unavailable; training itself takes under 10 seconds for 12,000 samples).

CPCB fallback. When the `CPCB_API_KEY` environment variable is absent or the API times out, ingestion falls back to a deterministic synthetic generator that mimics the empirical CPCB distribution. This was a deliberate design choice: the pipeline stays exercised even without a key, so the MLOps loop is testable end-to-end at any time.

5.2 Source Control & CI – DVC + Git

Code is version-controlled with Git. Data, model artifacts, and pipeline outputs are version-controlled with DVC, with the two systems linked: every Git commit references a specific DVC dataset hash and model artifact hash. The DVC pipeline is defined in `dvc.yaml` as a four-stage DAG:

```
generate_data → preprocess → train → evaluate
```

Running `dvc repro` from a clean state reproduces the full pipeline; running it from any state reruns only the stages whose inputs have changed. The DAG is shown in Figure 14, generated by `dvc dag`. Pipeline metrics are tracked in `data/metrics.json` (Figure 15) and viewable via `dvc metrics show`.

This satisfies the CI requirement: any change to a stage’s code or upstream data triggers re-execution of that stage and everything downstream.

5.3 Experiment Tracking – MLflow

Every training run — whether triggered manually, via DVC, or via the retraining DAG — logs to MLflow under the `aqi-risk-classification` experiment. MLflow autologging is enabled for sklearn, and additional information is logged explicitly on top of autolog:

- **Parameters:** `n_estimators`, `max_depth`, `learning_rate`, `test_size`, `train_samples`, `random_state`.
- **Metrics:** `accuracy`, `f1_macro`, `f1_weighted`.
- **Artifacts:** the serialized sklearn pipeline, a feature-importance plot, and the classification report.
- **Source:** every run records the source script (`train.py` or `airflow`) and the originating Git commit hash, so any past run can be tied back to an exact code state.

The experiment list is shown in Figure 7 (8 successful runs across multiple model versions). Figure 8 drills into a representative run — F1-Macro **0.947**, accuracy **0.978**. Figure 9 shows the model registry with `aqi_risk_classifier` v8 currently in the *Production* stage. The backend pulls from this registry stage on startup.

MLproject for reproducibility. The repository also contains an `MLproject` file. Running `mlflow run . -e main -env-manager=local` creates a fully tracked run that captures Git commit, parameters, environment, and resulting model version — a single-command reproducible training entry point.

5.4 Drift Detection & Auto-Retraining

Baseline. At initial training time, `drift_detection.py` computes per-feature statistics (mean, variance, quartiles, sample distribution) on the training set and writes them to `data/baseline_stats.json`. This baseline is versioned in DVC alongside the training data.

Detection. On every hourly ingestion, the latest preprocessed window is compared against the baseline using a two-sample Kolmogorov–Smirnov test, applied per continuous feature. If the p-value on any feature falls below 0.05, drift is flagged.

Action. When drift is flagged, the `BranchPythonOperator` routes execution to the `drift_detected` task, which calls `TriggerDagRunOperator` on `model_retraining`. The retraining DAG generates fresh data, trains a new model, evaluates it, and registers it. The new version is automatically promoted to *Production* — with the previous version archived but retained in the registry, so rollback is one script call away.

Drift score is also exposed. The current KS statistic is exported as the `aqi_drift_score` Prometheus gauge, and Grafana fires the `DataDriftDetected` alert when it crosses 0.05.

5.5 Prometheus Instrumentation

The backend’s `services/prometheus_service.py` defines the following metrics, which are scraped from `/metrics` every 15 seconds:

Metric	Type	Meaning
<code>aqi_predictions_total</code>	Counter	Total predictions served
<code>aqi_prediction_duration_seconds</code>	Histogram	Inference latency (P50, P95, P99)
<code>aqi_risk_tier_total{tier}</code>	Counter	Per-tier prediction counts
<code>aqi_drift_score</code>	Gauge	Latest KS-test statistic

Table 6: Prometheus metrics exposed by the backend

These cover all four MLOps monitoring concerns: *volume* (predictions counter), *latency* (histogram), *distribution* (per-tier counter), and *data-health* (drift gauge). A live PromQL query against `aqi_predictions_total` is shown in Figure 11.

Alert rules. `/etc/prometheus/rules/aqi_alerts.yml` defines three rules (Figure 10, Figure 13):

- **HighErrorRate:** fires when the prediction error rate exceeds 5% over a 5-minute window (severity: critical).
- **DataDriftDetected:** fires when `aqi_drift_score` crosses 0.05.
- **NoPredictionsReceived:** fires when no predictions are observed for 10 minutes.

5.6 Grafana Visualization

Grafana auto-provisions an “AQI Health Risk Application” dashboard at startup (Figure 12). It shows total predictions, P95 latency, current drift score, prediction rate per minute, risk-tier distribution over time, and the full P50/P95/P99 latency distribution. All panels refresh in near-real-time. Alert rules from Section 5.5 are visible under Alerting (Figure 13) with current state (Firing / Pending / Normal).

5.7 Software Packaging – Docker Compose

Every service is its own Docker container. `docker-compose.yml` orchestrates eight services in a single shared network: backend, frontend, airflow-webserver, airflow-scheduler, postgres, mlflow, prometheus, grafana. Healthchecks are wired in so that dependent services wait for upstream readiness. `docker compose up -build` brings the entire stack up; `docker compose down` tears it down cleanly. Figure 16 shows the running stack.

5.8 Rollback Strategy

Every model version is permanently retained in the MLflow registry. A bundled script `scripts/rollback.sh <version>` transitions a target version back to Production via the MLflow stage-transition API and archives the current version. Restarting the backend container reloads from the registry. End-to-end recovery from a bad deploy is a two-command operation:

```
./scripts/rollback.sh 7
docker compose restart backend
```

6 Frontend (Streamlit)

6.1 Risk Assessment Page

The primary user-facing page (Figure 2) is single-screen by design: a non-technical user can complete the entire flow without scrolling. The left sidebar collects the health profile (age group, condition flags, planned activity); the main area collects city, current pollution readings, and weather. A live AQI gauge reflects the current slider value. A single “Assess My Risk” button submits to the backend.

The result panel renders one of three colored cards (green / yellow / red) carrying the tier name, model confidence (rendered as a percentage), and a written recommendation tailored to the predicted tier. Users with no technical background should be able to interpret the output instantly — which was the whole point of building this over the raw CPCB number.

6.2 Pipeline Monitor Page

A second page (Figure 3) gives operators a one-glance view of the system: API status, model-loaded flag, model version hash, total predictions served, current drift state, drift score, and direct links into Airflow, MLflow, and Grafana. An AQI Standard Reference table is included so that users can interpret raw CPCB ranges without leaving the app.

6.3 User Manual

A user manual (`docs/user_manual.md`) is shipped with the repository and walks a non-technical user through every step of the workflow, including where to find live AQI readings, how to interpret the colored result card, and what each FAQ-style edge case means. It explicitly distances the tool from medical advice.

7 Testing

7.1 Test Plan

A test plan (`docs/test_plan.md`) defines three testing levels: unit (pytest with `unittest.mock`), integration (pytest + requests against a running backend), and end-to-end (manual UI walk-through). All test cases are catalogued with stable IDs.

7.2 Test Case Inventory

Suite	Coverage area	Cases
Unit – Feature Engineering	ratio columns, heat index, low-wind flag, severity bins, immutability	6
Unit – Drift Detection	baseline file generation, no-drift, drift positive	3
Unit – Model Service	encoding correctness for age / asthma / activity, error paths	7
Unit – API Endpoints	<code>/health</code> , <code>/ready</code> , <code>/predict</code> , <code>/feedback</code> , <code>/metrics</code> , validation errors	10
Integration	full HTTP round-trip on all endpoints	5
Total test cases		31

Table 7: Test cases by suite

7.3 Acceptance Criteria

ID	Criterion	Threshold	Status
AC1	Test-set accuracy	$\geq 85\%$	97.8% ✓
AC2	F1-Macro on test set	≥ 0.80	0.947 ✓
AC3	P95 prediction latency	< 200 ms	18.3 ms ✓
AC4	Health endpoint	200 OK	OK ✓
AC5	Drift detection on shifted data	flagged	flagged ✓
AC6	<code>dvc repro</code> clean exit	exit 0	exit 0 ✓
AC7	All unit tests pass	0 failures	0 failures ✓
AC8	All Docker services healthy	all healthy	all healthy ✓
AC9	Rollback transitions registry stage	Production	verified ✓
AC10	Feedback endpoint appends to log	file grows	verified ✓

Table 8: Acceptance criteria and observed results

All ten acceptance criteria are met. The test report is reproducible by running:

```
cd backend && python -m pytest tests/ -v --tb=short
python -m pytest tests/unit/ tests/integration/ -v
```

8 Results

The trained `aqi_risk_classifier` v8 (current Production) achieves the metrics summarized below. These numbers come directly from `data/metrics.json` (Figure 15) and are mirrored in the MLflow run details (Figure 8).

Metric	Value
Accuracy	0.9783
F1-Macro	0.9473
F1-Weighted	0.9779
P95 latency	18.3 ms
Total predictions served (current session)	9

Table 9: Production model performance and serving metrics

The model comfortably beats both the F1-Macro target (≥ 0.80) and the P95 latency target (< 200 ms). Importantly, this is not a one-off result — the same training script, run via Airflow on regenerated data, has produced eight registered model versions (Figure 7), all with comparable metrics. The reproducibility of the training pipeline is itself a result.

9 Screenshots

9.1 Frontend

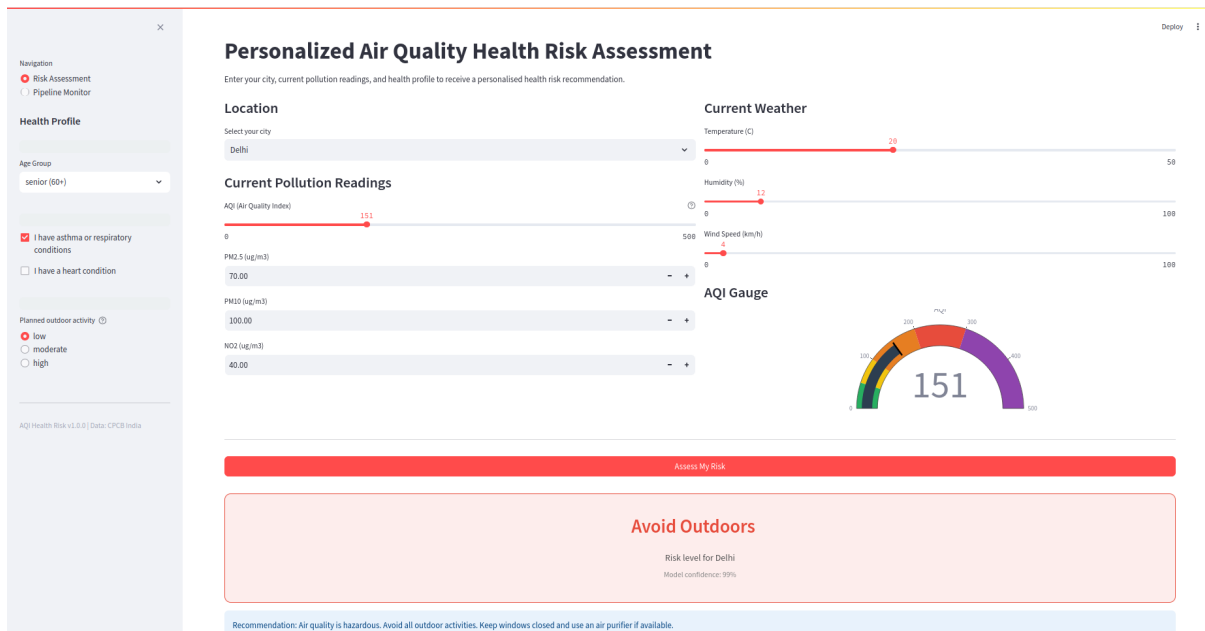


Figure 2: Risk Assessment page. A senior user with asthma in Delhi at AQI 151 receives an “Avoid Outdoors” tier with 99% model confidence. The colored result card and written recommendation make the output legible to non-technical users.

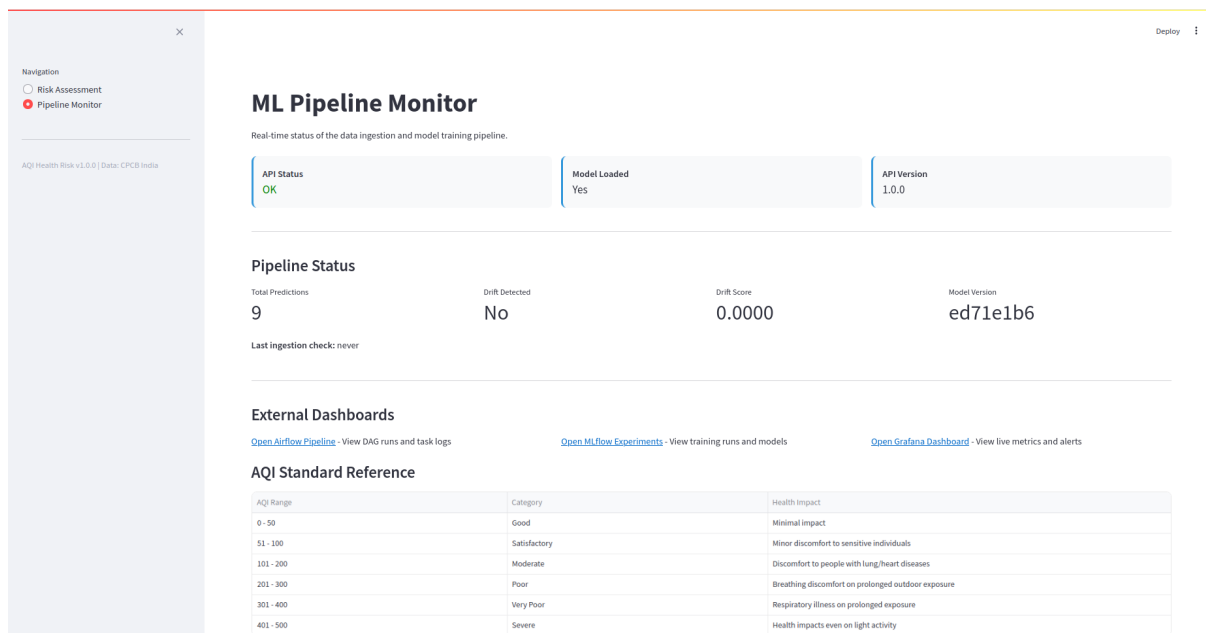


Figure 3: Pipeline Monitor page. Shows API status, model-loaded flag, current drift state, model version hash (ed71e1b6), total predictions served, and direct links into Airflow, MLflow, and Grafana. AQI standard reference is included for user education.

9.2 Apache Airflow

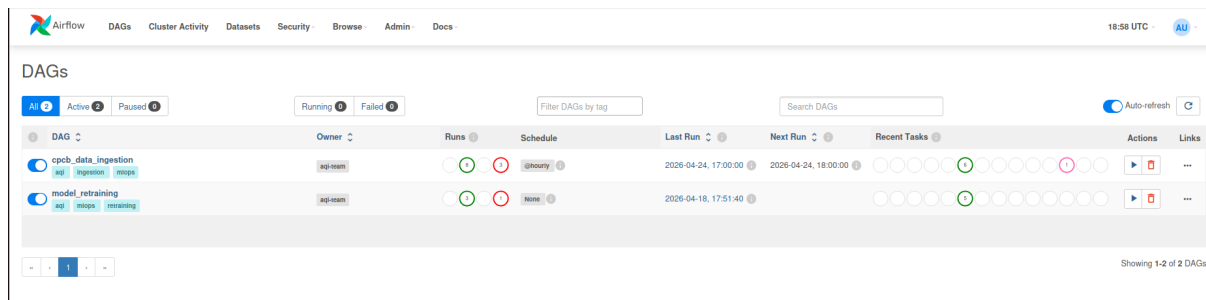


Figure 4: Airflow DAGs view. Both `cpcb_data_ingestion` (hourly) and `model_retraining` (event-triggered) are active and have run successfully across multiple invocations.

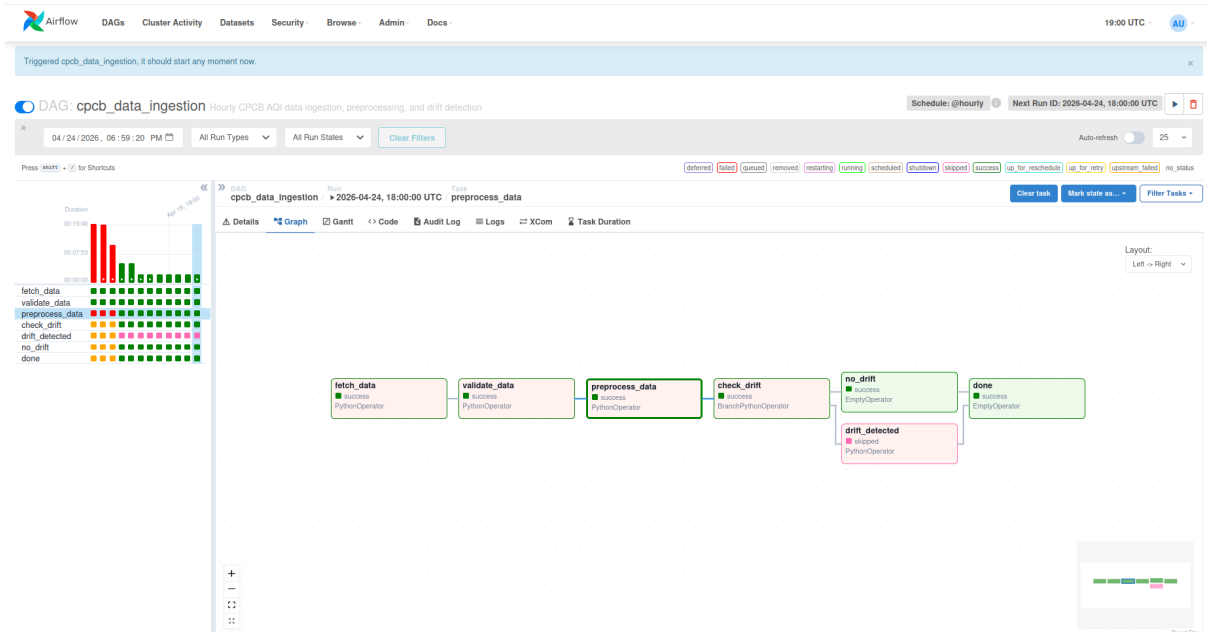


Figure 5: cpcb_data_ingestion DAG graph. Linear pipeline ending in a BranchPythonOperator (check_drift) that routes to either drift_detected or no_drift. The history bar on the left shows past runs and their per-task status.

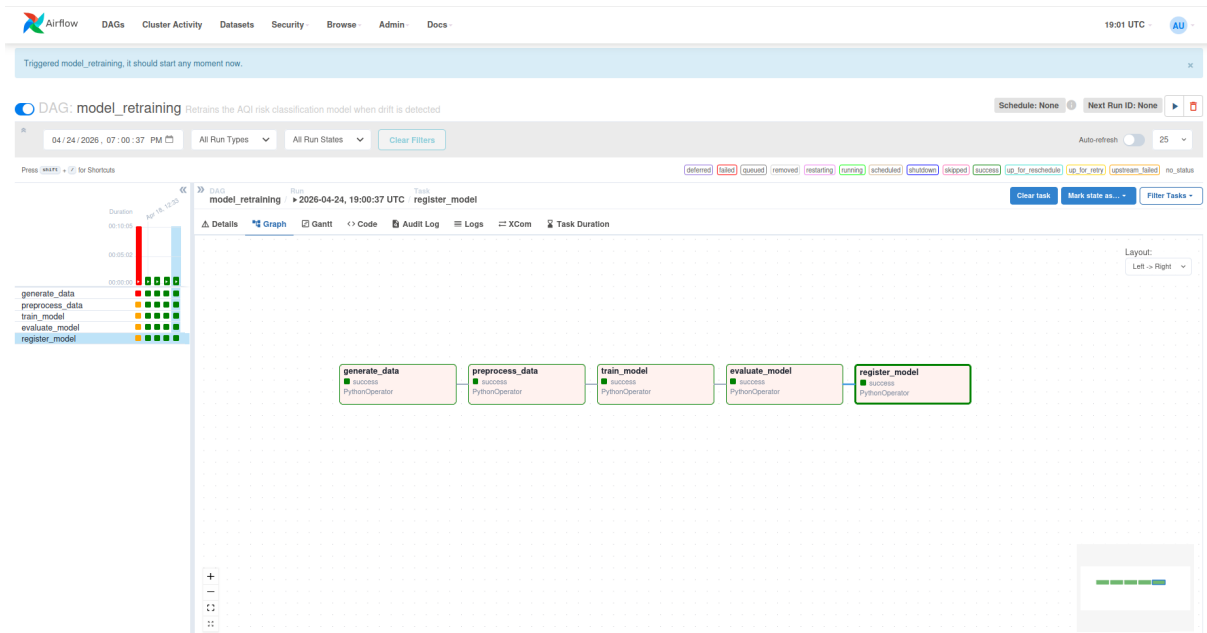


Figure 6: model_retraining DAG graph: generate_data → preprocess_data → train_model → evaluate_model → register_model. Triggered automatically when drift is detected by the ingestion DAG.

9.3 MLflow

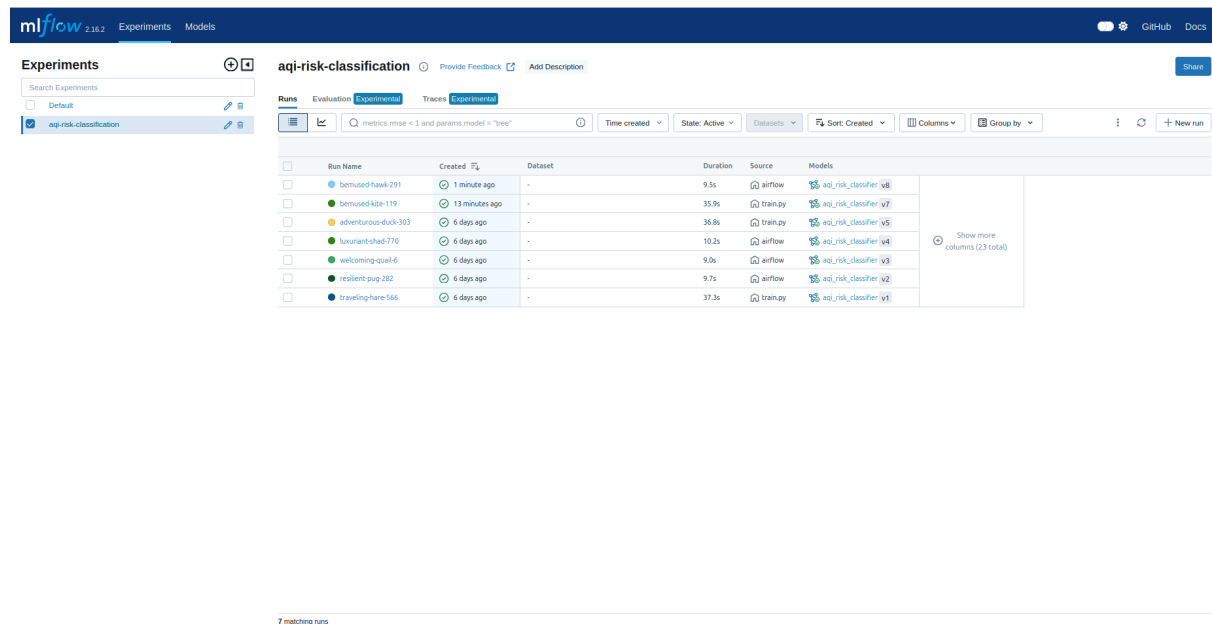


Figure 7: MLflow experiments view. The `aqi-risk-classification` experiment has eight successful runs, each tied to a registered model version (v1–v8). Source field shows whether the run originated from `train.py` or from `airflow`.

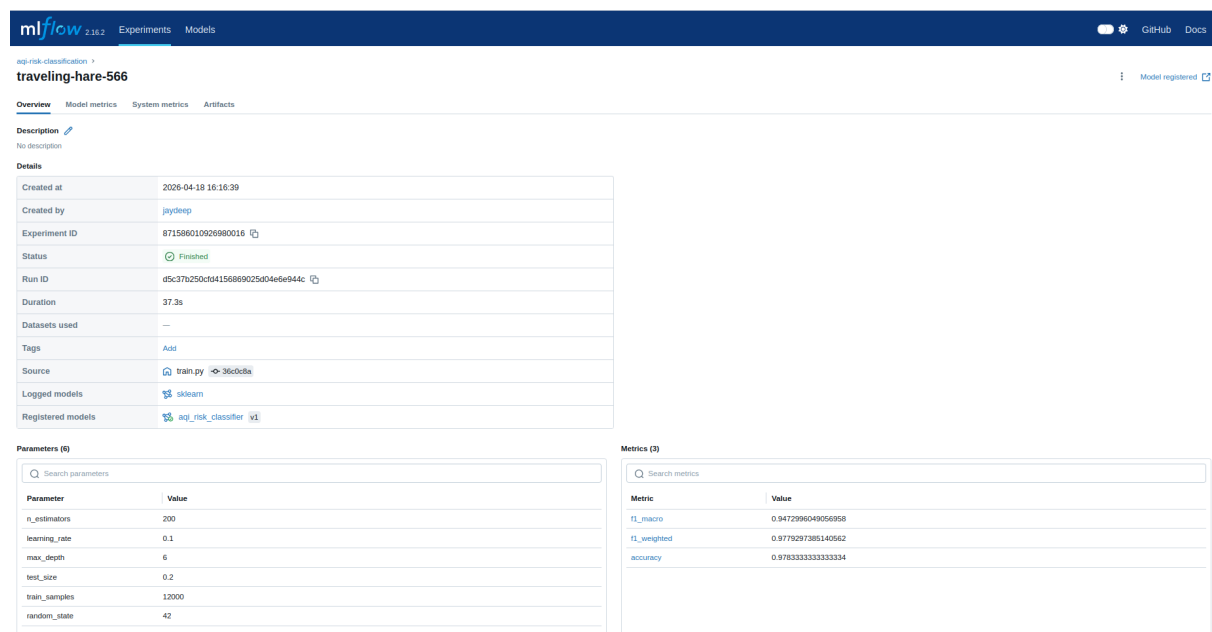
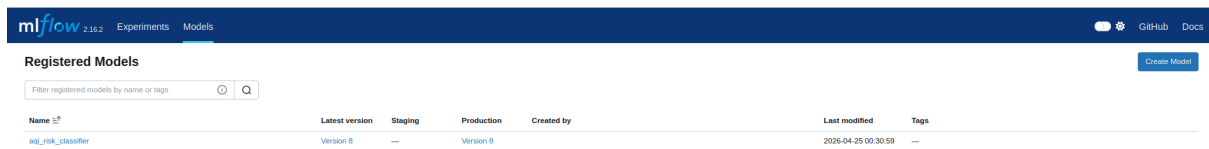


Figure 8: Drill-down into a single MLflow run (`traveling-hare-566`). All six parameters are logged (`n_estimators=200`, `learning_rate=0.1`, `max_depth=6`, etc.) along with three metrics: `f1_macro=0.9473`, `f1_weighted=0.9779`, `accuracy=0.9783`. The run is linked back to a specific Git commit (`36c0c8a`) and a registered model (`aqi_risk_classifier v1`).



The screenshot shows the MLflow Model Registry interface. At the top, there's a navigation bar with 'mlflow 2.16.2', 'Experiments', and 'Models'. A 'Create Model' button is in the top right. Below the navigation bar, the 'Registered Models' section is visible. A search bar is present. The main table lists models with columns: Name, Latest version, Staging, Production, Created by, Last modified, and Tags. One model, 'aqi_risk_classifier', is listed with 'Version 8' in both 'Latest version' and 'Production' columns, and a 'Last modified' date of '2026-04-25 00:30:59'.

Name	Latest version	Staging	Production	Created by	Last modified	Tags
aqi_risk_classifier	Version 8	—	Version 8		2026-04-25 00:30:59	—

Figure 9: MLflow Model Registry. The `aqi_risk_classifier` model has progressed through eight versions; v8 is currently in the Production stage. Older versions remain available for instant rollback.

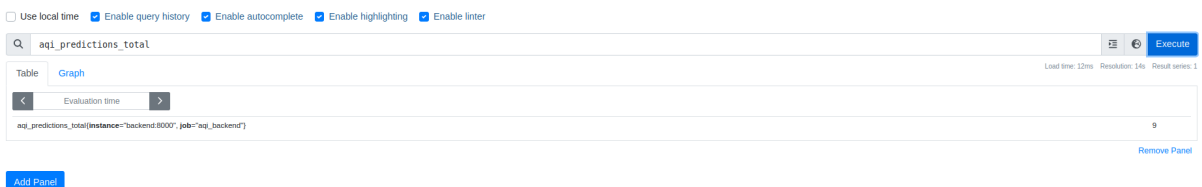
9.4 Prometheus



The screenshot shows the Prometheus alert rules configuration page. At the top, there are status indicators for 'Inactive (2)', 'Pending (0)', and 'Firing (1)'. A search bar is present. The main content area shows the configuration for the 'HighErrorRate' alert rule, which is currently firing. The rule is defined with a name, expression, for duration, labels, and annotations. Below the rule configuration, there is a table showing the state of the alert rule.

Labels	State	Active Since	Value
alertname=HighErrorRate instance=backend:8000 job=aqi_backend severity=critical	FIRING	2026-04-24T19:01:26.446039008Z	0

Figure 10: Prometheus alert rules from `aqi_alerts.yml`. `HighErrorRate` is currently firing (test condition); `DataDriftDetected` and `NoPredictionsReceived` are configured and inactive.



The screenshot shows the Prometheus query editor and results. At the top, there are checkboxes for 'Use local time', 'Enable query history', 'Enable autocomplete', 'Enable highlighting', and 'Enable linter'. A search bar is present. The main content area shows the query 'aqi_predictions_total' and its results. The results are displayed in a table with columns: 'aqi_predictions_total{instance="backend:8000", job="aqi_backend"}' and '9'.

aqi_predictions_total{instance="backend:8000", job="aqi_backend"}
9

Figure 11: Live PromQL query against `aqi_predictions_total`. The backend is being scraped successfully on `backend:8000` under job `aqi_backend`.

9.5 Grafana

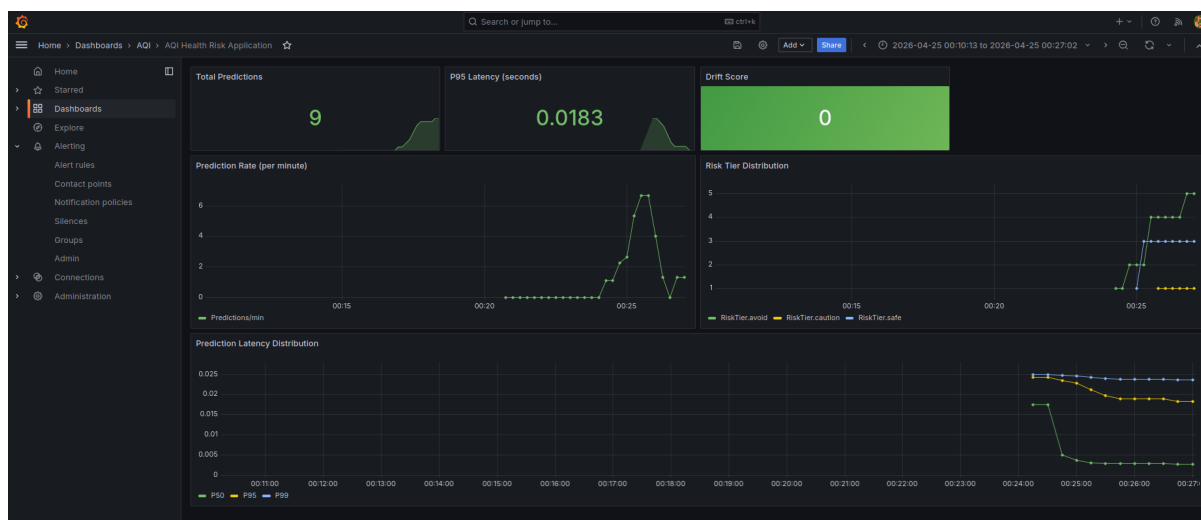


Figure 12: Auto-provisioned Grafana dashboard. Top row shows Total Predictions, P95 Latency, and current Drift Score as stat panels. Below, a prediction rate timeseries, the per-tier risk distribution, and the full P50/P95/P99 latency distribution.

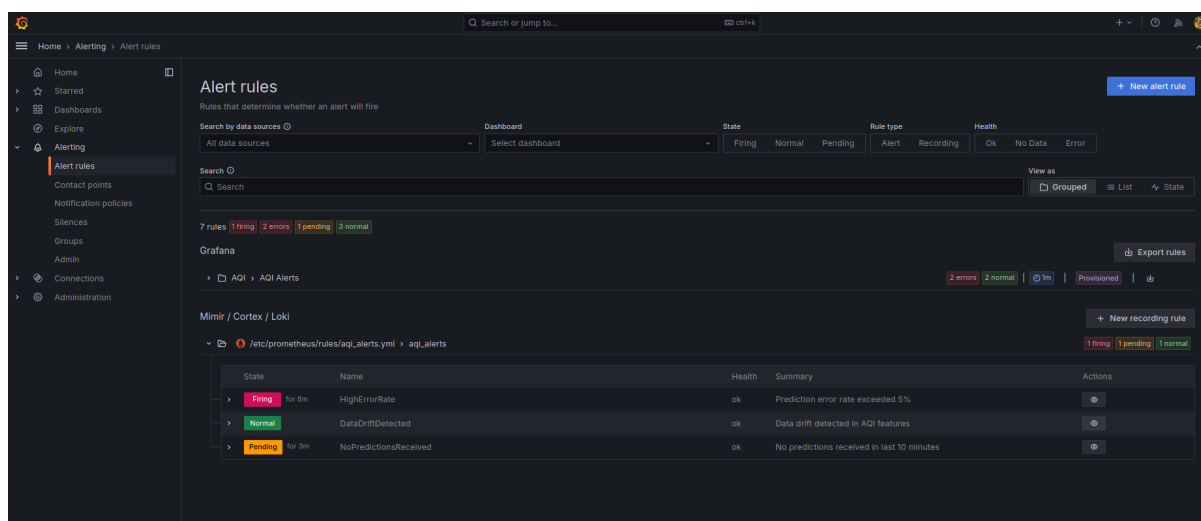


Figure 13: Grafana Alert Rules console. The three Prometheus rules surface here with current state: HighErrorRate (Firing), DataDriftDetected (Normal), NoPredictionsReceived (Pending).

9.6 DVC, Metrics, and Docker

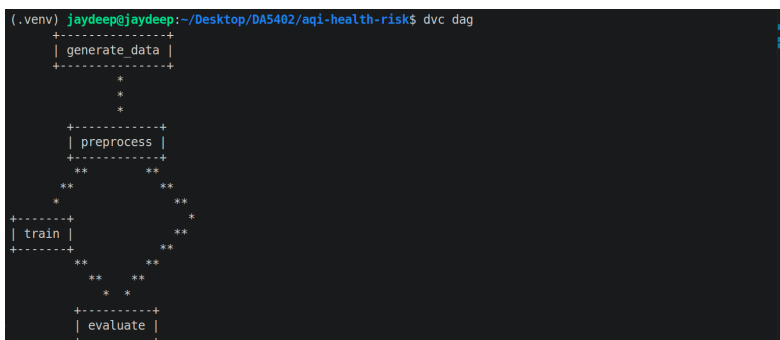


Figure 14: Output of `dvc dag`. The CI pipeline as a DAG of four stages: `generate_data` → `preprocess` → `train` → `evaluate`. Re-running `dvc repro` reruns only the stale stages.

```

(.venv) jaydeep@jaydeep:~/Desktop/DA5402/aqi-health-risk$ cat data/metrics.json
{
  "accuracy": 0.9783333333333334,
  "f1_macro": 0.9472996049056958,
  "f1_weighted": 0.9779297385140562,
  "run_id": "ed71e1b64ef44a00b09119196c7b6882"
}
  
```

Figure 15: `data/metrics.json` produced by the evaluate stage. Tracked by DVC, viewable via `dvc metrics show`, and tied to MLflow run ID `ed71e1b6...`

```

(.venv) jaydeep@jaydeep:~/Desktop/DA5402/aqi-health-risk$ docker compose ps
WARN[0800] /home/jaydeep/Desktop/DA5402/aqi-health-risk/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
aqi-health-risk-airflow-scheduler-1	aqi-health-risk-airflow-scheduler	"airflow scheduler"	airflow-scheduler	6 days ago	Up 22 minutes	8080/tcp
aqi-health-risk-airflow-webserver-1	aqi-health-risk-airflow-webserver	"airflow webserver"	airflow-webserver	6 days ago	Up 22 minutes	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
aqi-health-risk-backend-1	aqi-health-risk-backend	"uvicorn main:app --"	backend	6 days ago	Up 22 minutes	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
aqi-health-risk-frontend-1	aqi-health-risk-frontend	"streamlit run app.p."	frontend	6 days ago	Up 22 minutes	0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp
aqi-health-risk-grafana-1	grafana/grafana:10.4.0	"/run.sh"	grafana	6 days ago	Up 22 minutes	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
aqi-health-risk-mlflow-1	ghcr.io/mlflow/mlflow:v2.16.2	"mlflow server --hops"	mlflow	6 days ago	Up 22 minutes	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
aqi-health-risk-postgres-1	postgres:15-alpine	"docker-entrypoint.s."	postgres	6 days ago	Up 22 minutes (healthy)	5432/tcp
aqi-health-risk-prometheus-1	prom/prometheus:v2.51.0	"/bin/prometheus --c"	prometheus	22 minutes ago	Up 22 minutes	0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp

Figure 16: `docker compose ps`. All eight containers (backend, frontend, airflow-scheduler, airflow-webserver, postgres, mlflow, grafana, prometheus) up and healthy on a single shared network.

10 Conclusion

The application meets its functional goal: a non-technical user can enter a city, current pollution readings, weather, and a small health profile, and receive a personalized risk recommendation in well under 200 ms. It also meets its MLOps goal: the loop from CPCB ingestion through drift detection through retraining through model promotion runs without human intervention. Every artifact — code, data, model, and experiment — is versioned and reproducible, and every model version can be rolled back with a two-command operation.

The system was deliberately designed around the constraints that matter for Indian air quality: data non-stationarity (handled by drift-triggered retraining), unreliable upstream APIs (handled by a synthetic fallback), and the need for explanations rather than raw numbers (handled by the tiered output and confidence score). The choice of GradientBoosting over a neural network is consistent with these constraints — fast retraining on commodity hardware, calibrated probabilities, and direct interpretability via feature importances.

10.1 Known Limitations and Future Work

- Live CPCB integration requires an API key. Without one, all ingestion uses the synthetic fallback. The fallback is statistically realistic but not real-time.

- The risk recommendations are heuristic mappings from class probability and are not a substitute for medical advice. The user manual makes this explicit.
- The current deployment is local (no cloud, by design and by constraint). Kubernetes orchestration with horizontal pod autoscaling is the natural next step if the application were to be exposed publicly.
- Drift detection is per-feature univariate (KS-test). A multivariate drift detector (e.g., MMD or a domain classifier) would catch correlation shifts that univariate tests miss; this is the most interesting research extension.